2048 as a Markov decision problem

Project work for *Dynamic Programming and Reinforcement Learning* (University of Siena, A.Y. 2024/25), by Edoardo Caproni and Tommaso Quintabà.

Watch the agents play in the browser: edoardocaproni.github.io/2048. In this policy viewer the best policy found by every method plays: on the 2x3 board with the exact optimal win probability of every position and move, on the original 4x4 game with the tabular methods and the deep Q-networks of the final test.

2048 is a puzzle for one player. Every move has two steps: the player slides all the tiles in one direction (deterministic), then a new tile appears in a random empty cell (random). The rules, including the probabilities of the new tile, are completely known, so the game is a Markov decision problem with a known model. Small versions of the game can be solved exactly by dynamic programming; the original 4x4 game cannot, because it has far too many states. The project is built around this gap:

- **Part 1, exact dynamic programming.** On small boards (up to 3x3 cells) we compute the optimal policy exactly, and use it to measure how good the usual rules of thumb are and how much of the game is luck.
- **Part 2, reinforcement learning.** We forget the model and learn from a simulator with the methods of the course. On the small boards every learned policy is measured exactly against the DP solution.

Contents

1 The game **1**

2 Problem formulation **2**

3 Methods **3**
3.1 Exact DP by stages . . . 3
3.2 Value iteration . . . 5
3.3 Policy evaluation and benchmark policies . . . 5
3.4 Choosing the reward . . . 6
3.5 Prediction: TD(0), n-step TD, Monte Carlo . . . 9
3.6 Tabular control . . . 9
3.7 Exploration . . . 12
3.8 Exact learning curves . . . 13
3.9 Final test: deep Q-network . . . 14
3.10 Extra: Sarsa with n-tuple features . . . 15

4 Main results **16**

5 Repository **17**
5.1 Setup . . . 17
5.2 Code . . . 17
5.3 Reproducing the results . . . 17

1 The game

The board has $R \times C$ cells and the target tile is 2^K (the original game: $R = C = 4$, $K = 11$, tile 2048).

1. A move (down, left, right, up) slides all the tiles as far as possible in that direction. Two equal tiles that collide merge into one tile carrying their sum; a tile created by a merge cannot merge again in the same move. The score grows by the value of every tile created by a merge.
2. A move is legal only if it changes the board.

3. After every legal move a new tile appears in a uniformly random empty cell: a 2 with probability $1-p_4$, a 4 with probability p_4 ($p_4 = 0.1$ in the original game). The game starts with two such tiles.
4. The game is won when a move creates the target tile, and lost when no legal move is left.

2 Problem formulation

We use the notation of the DP part of the course (Bertsekas): state x, control u, disturbance w, dynamics $x_{t+1} = f(x_t, u_t, w_t)$. The RL part switches to the notation of Sutton & Barto, as the course does. Every symbol is explained where it first appears.

State. A board is stored as the exponents of its tiles, cell by cell: $x = (x_1, \dots, x_{RC})$, with $x_i = 0$ for an empty cell and $x_i = k$ for the tile 2^k. The index $t = 0, 1, 2, \dots$ counts the moves, and the state x_t is the board the player sees before move t, after the random tile.

Control. The control u is the move. $U(x)$ is the set of legal moves of the board x, the ones that change it:

$$U(x) = \{u \in \{\text{down}, \text{left}, \text{right}, \text{up}\} : M(x, u) \neq x\}.$$

The game is over exactly when $U(x)$ is empty.

Afterstate (deterministic part of a move). $y = M(x, u)$ is the board after the slide and the merges, before the new tile. It is the part of the next board that the player controls. The points of the move, $r(x, u)$, are the sum of the tiles created by merges.

Disturbance (random part of a move). $w_t = (i_t, v_t)$: the cell i_t of the new tile, uniform over the set $E(y_t)$ of the empty cells of the afterstate (there are $|E(y_t)|$ of them), and its exponent v_t, equal to 1 (a 2) with probability $1 - p_4$ and 2 (a 4) with probability p_4. Given the afterstate, the new tile does not depend on anything else.

Dynamics. Slide, then add the random tile. Here e_i is the board with a 1 in cell i and 0 elsewhere, so $y + v\, e_i$ is the board y with the exponent v placed in the empty cell i:

$$x_{t+1} = f(x_t, u_t, w_t) = M(x_t, u_t) + v_t\, e_{i_t}.$$

Objective. Maximise the probability of creating the target tile:

$$V^*(x) = \max_{\pi} \; P^{\pi}\left(\text{the target tile is created} \mid x_0 = x\right).$$

Read it from the inside out. A policy π is a rule that picks the move on every board. $P^{\pi}(\cdots \mid x_0 = x)$ is the probability of winning when the game starts from the board x and the player chooses every move with the policy π: the moves follow the rule, the new tiles are random, and the probability is taken over the new tiles. The maximum is over all the policies, so $V^*(x)$ is the win probability of the best possible player starting from x. Throughout, a star marks a value of the best policy ("optimal").

In cost form this is a problem with no cost per stage and a terminal cost of 1 for a lost game and 0 for a won game; the optimal cost is $J^* = 1 - V^*$, the smallest probability of losing. For comparison we also study the expected score, with the points $r(x, u)$ as the reward of each stage.

Bellman equation. Each move has two steps: first the player chooses a slide, then chance places the new tile. We give a value to the board at both moments:

- $V^*(x)$ is the win probability of the best player from the board x, the moment *before the player chooses* a move;
- $W^*(y)$ is the win probability of the best player from the afterstate y, the moment *after the slide and before the new tile*. The player has already moved, so here nothing is chosen: the next thing that happens is the random tile, and after it the player sees a new board x' and plays its best from there. So $W^*(y)$ is the average of V^* over the boards x' that the new tile can produce from y.

W^* is not a new quantity of the game, only a name for this intermediate average. With it the two steps become two short equations, one for the choice (a maximum) and one for the chance (an average):

$$V^*(x) = \begin{cases} 0 & \text{if } U(x) = \emptyset, \\ \max_{u \in U(x)} W^*\big(M(x,u)\big) & \text{otherwise,} \end{cases}$$

$$W^*(y) = \begin{cases} 1 & \text{if } \max_i y_i = K, \\ \displaystyle\sum_{i \in E(y)} \frac{1}{|E(y)|} \Big[(1-p_4)\, V^*(y + e_i) + p_4\, V^*(y + 2e_i) \Big] & \text{otherwise.} \end{cases}$$

In words: the player picks the move whose afterstate is worth most, and a board with no legal move is lost. An afterstate holding the target ($\max_i y_i = K$: the largest exponent on the board is K, the tile 2^K) is a win. Otherwise the new tile goes to each of the $|E(y)|$ empty cells with probability $1/|E(y)|$, and is a 2 (exponent 1, board $y + e_i$) with probability $1 - p_4$ or a 4 (exponent 2, board $y + 2e_i$) with probability p_4; $W^*(y)$ is the average of V^* over these boards, each weighted by its probability.

Bellman equation for the expected score. The same two steps, with values that count points instead of probabilities: $V^*_{\text{score}}(x)$ is the largest expected number of points still to come from the board x, before the move, and $W^*_{\text{score}}(y)$ the same from the afterstate y, before the new tile:

$$V^*_{\text{score}}(x) = \begin{cases} 0 & \text{if } U(x) = \emptyset, \\ \max_{u \in U(x)} \Big[r(x,u) + W^*_{\text{score}}\big(M(x,u)\big) \Big] & \text{otherwise,} \end{cases}$$

$$W^*_{\text{score}}(y) = \sum_{i \in E(y)} \frac{1}{|E(y)|} \Big[(1-p_4)\, V^*_{\text{score}}(y + e_i) + p_4\, V^*_{\text{score}}(y + 2e_i) \Big].$$

Two things change. The player now picks the move with the most points now plus the most expected points later, $r(x,u) + W^*_{\text{score}}$, instead of the move with the best afterstate alone. And the case "the afterstate holds the target" is gone: creating the target does not end the game, which goes on until no legal move is left. A game over still gives 0, because no more points can be made. The tile sum is still a clock, so the same backward pass solves this equation too.

The key property: the tile sum is a clock. Let $\sigma(x) = \sum_{i:x_i>0} 2^{x_i}$ be the sum of the tiles. A slide never changes it (a merge turns $2^k + 2^k$ into 2^{k+1}), and the new tile adds 2 or 4:

$$\sigma\big(M(x,u)\big) = \sigma(x), \qquad \sigma(x_{t+1}) \in \{\sigma(x_t) + 2,\ \sigma(x_t) + 4\}.$$

Consequences:

- no board is visited twice in a game, and every game ends within $RC \cdot 2^{K-2}$ moves, whatever the policy. So every policy is proper (it ends the game for sure), and the Bellman equation has a unique solution;
- grouping the boards by tile sum into stages X_σ, every move goes from X_σ to $X_{\sigma+2}$ or $X_{\sigma+4}$: the problem can be solved by a single backward pass, like a finite-horizon problem.

3 Methods

3.1 Exact DP by stages

1. **Forward pass:** starting from the two initial tiles, generate the reachable boards stage by stage, for increasing tile sum.
2. **Backward pass:** for σ from the largest to the smallest, and for every board $x \in X_\sigma$, compute W^* of each afterstate with the Bellman equation and set $V^*(x) = \max_u W^*(M(x,u))$.

When a board is processed, every board it can lead to has a larger tile sum and already has its final value, so one pass is enough. The cost is proportional to (number of states) x (moves) x (possible new tiles). This solves the 2x3 board with target 64 (27,022 states) in a fraction of a second and the 3x3 board with target 64 (4.8 million states) in four minutes, in pure Python.

In the code. A board is a tuple of exponents, and `game.moves(board)` lists the legal moves as triples `(action, after, points)`: the move u, the afterstate $y = M(x,u)$ and the points $r(x,u)$. The weights of the average in W^*, $(1-p_4)/|E(y)|$ and $p_4/|E(y)|$, come from the distribution of the new tile:

Code: src/game.py, lines 125-137: `Game2048.spawn_distribution`

```
def spawn_distribution(self, afterstate):
    """Distribution of the next state: list of (board, probability).

    After a legal move there is always at least one empty cell.
    """
    empty = [i for i, e in enumerate(afterstate) if e == 0]
    out = []
    for i in empty:
        for e, p in self._spawn_values:
            board = list(afterstate)
            board[i] = e
            out.append((tuple(board), p / len(empty)))
    return out
```

The afterstate equation, $W^*(y)$: 1 if y holds the target (win objective only), otherwise the average of V over the next boards:

Code: src/dynamic_programming.py, lines 94-98: `afterstate_value`

```
def afterstate_value(game, after, V, objective='win'):
    """W(y): value of an afterstate y, before the random tile appears."""
    if objective == 'win' and game.is_win(after):
        return 1.0
    return sum(p * V[nxt] for nxt, p in game.spawn_distribution(after))
```

The forward pass groups the reachable boards by tile sum (`layers[sigma]` is the stage X_σ):

Code: src/dynamic_programming.py, lines 62-77: `reachable_layers`

```
layers = defaultdict(set)
for board, _ in game.initial_distribution():
    layers[game.tile_sum(board)].add(board)
sigma = min(layers)
while sigma <= max(layers):
    for board in layers.get(sigma, ()):
        for _, after, _ in game.moves(board):
            if stop_at_target and game.is_win(after):
                continue
            # the new tile adds exponent 1 (a 2) or 2 (a 4), i.e. the
            # tile sum goes from sigma to sigma + 2 or sigma + 4
            base = sum(after)
            for nxt, _ in game.spawn_distribution(after):
                layers[sigma + 2 ** (sum(nxt) - base)].add(nxt)
    sigma += 2
return dict(layers)
```

The backward pass solves both Bellman equations. `r` is `None` for the win objective, so `q` is $W^*(M(x,u))$; for the score it is $r(x,u) + W^*_{\text{score}}(M(x,u))$. A board with no legal move keeps the value 0:

Code: src/dynamic_programming.py, lines 107-125: `solve_optimal`

```
layers = _default_layers(game, objective, layers)
r = _stage_reward(game, objective)
V, table = {}, {}
for sigma in sorted(layers, reverse=True):
    W = {}  # afterstates of the boards of stage sigma have tile sum sigma
    for board in layers[sigma]:
        best_action, best_value = None, 0.0  # game over: value 0
        for action, after, points in game.moves(board):
            w = W.get(after)
            if w is None:
                w = W[after] = afterstate_value(game, after, V, objective)
            q = w if r is None else w + r(board, after, points)
            if best_action is None or (q > best_value and
                                       not _is_tie(q, best_value)):
                best_action, best_value = action, q
        V[board] = best_value
        if best_action is not None:
            table[board] = best_action
return V, table, layers
```

3.2 Value iteration

Start from $V^{(0)} = 0$ and apply the Bellman equation to all the states at every iteration,

$$V^{(i+1)}(x) = \max_{u \in U(x)} W^{(i)}\big(M(x,u)\big),$$

where the superscript (i) is the iteration number and $W^{(i)}$ is the afterstate equation computed with $V^{(i)}$. Started from zero, $V^{(i)}(x)$ is the best probability of winning within i moves. Since every game is bounded, value iteration is exact after finitely many iterations (46 on the 2x3 board with target 64), not only in the limit. The backward pass above is the same iteration with the states updated in place (Gauss-Seidel) in decreasing tile sum: its first sweep already gives V^*.

In the code. `V` starts at 0 on every state. With `order='jacobi'` the new values go into a new table `new`, $V^{(i+1)}$, computed only from the old one, $V^{(i)}$; with `order='backward'`, `new` is `V` itself and the update is in place. `delta` is the largest change, $\max_x |V^{(i+1)}(x) - V^{(i)}(x)|$, and the loop stops when it is 0:

Code: src/dynamic_programming.py, lines 187-215: `value_iteration`

```
states = [b for sigma in sorted(layers, reverse=True) for b in layers[sigma]]
V = dict.fromkeys(states, 0.0)
table = {}
history = []
for it in range(1, max_iter + 1):
    new = V if order == 'backward' else {}
    W = {}  # afterstate values from the current V (cached per sweep)
    delta = 0.0
    for board in states:
        best_action, best = None, 0.0  # game over: value 0
        for action, after, points in game.moves(board):
            w = W.get(after)
            if w is None:
                w = W[after] = afterstate_value(game, after, V, objective)
            q = w if r is None else w + r(board, after, points)
            if best_action is None or (q > best and not _is_tie(q, best)):
                best_action, best = action, q
        delta = max(delta, abs(best - V[board]))
        new[board] = best
        if best_action is not None:
            table[board] = best_action
    V = new
    row = {'iteration': it, 'delta': delta, 'start': start_value(game, V)}
    if V_ref is not None:
        row['error'] = max(abs(V[b] - V_ref[b]) for b in states)
    history.append(row)
    if delta <= tol:
        break
return V, table, history
```

3.3 Policy evaluation and benchmark policies

Replacing the maximisation with the move chosen by a fixed policy μ (the DP part of the course writes μ for a policy; the RL part writes π) gives the exact value of that policy, with the same backward pass:

$$V_\mu(x) = W_\mu\big(M(x, \mu(x))\big).$$

Here $\mu(x)$ is the move the policy plays on the board x, and W_μ is the afterstate equation with V_μ in place of V^*.

We use it to evaluate three rules of thumb exactly: the **corner rule** (the first legal move in the order down, left, right, up: keep the big tiles in a corner), the **greedy rule** (the move with the most points) and **random play**. In the RL part, the same evaluation measures every learned policy.

In the code. The backward pass of `solve_optimal`, with the maximisation replaced by the moves of the policy. A policy returns `{action: probability}`, so the same code also evaluates random play, $V_\mu(x) = \sum_u \mu(u \mid x)\, W_\mu(M(x,u))$:

Code: src/dynamic_programming.py, lines 133-147: `evaluate_policy`

```
V = {}
for sigma in sorted(layers, reverse=True):
    W = {}
    for board in layers[sigma]:
        moves = game.moves(board)
        effect = {action: (after, points) for action, after, points in moves}
        value = 0.0
        for action, prob in policy(board, moves).items():
            after, points = effect[action]
            w = W.get(after)
            if w is None:
                w = W[after] = afterstate_value(game, after, V, objective)
            value += prob * (w if r is None else w + r(board, after, points))
        V[board] = value
return V
```

The three rules of thumb (`moves` follows the order down, left, right, up):

Code: src/dynamic_programming.py, lines 238-253: `corner_policy`, `greedy_policy`, `random_policy`

```
def corner_policy(board, moves):
    """Fixed priority down > left > right > up: the classic rule 'keep the
    big tiles in the bottom-left corner'."""
    return {moves[0][0]: 1.0} if moves else {}


def greedy_policy(board, moves):
    """Move with the largest immediate merge score (ties: ACTIONS order)."""
    if not moves:
        return {}
    return {max(moves, key=lambda m: m[2])[0]: 1.0}


def random_policy(board, moves):
    """Uniformly random legal move."""
    return {action: 1.0 / len(moves) for action, _, _ in moves}
```

3.4 Choosing the reward

A learning agent only knows what we want through its reward. Our first idea was to reward what good players do. For a move (x, u) with afterstate $y = M(x, u)$:

$$r_{\text{naive}}(x,u) = \underbrace{2^{\max_i y_i}\,\mathbf{1}\big[\max_i y_i > \max_i x_i\big]}_{\text{new largest tile}} + \underbrace{(2+2p_4)}_{\text{growth of the tile sum}} + \underbrace{\big(|E(y)| - |E(x)| - 1\big)}_{\text{change in free cells}}.$$

The first part pays the value of the largest tile when the move creates a new largest tile ($\mathbf{1}[\cdot]$ is 1 if the condition holds, 0 otherwise); the second is the average growth of the tile sum, 2 or 4; the third is the change in the number of empty cells from x to the next board (the new tile fills one, hence the -1). Adding up each part over a game shows what it really asks for:

- **tile sum:** by the clock property the move never changes the sum, so this part pays the same for every move: it only rewards surviving;
- **free cells:** the changes add up to $-|E(x_0)|$, because a lost game ends on a full board. It is a constant, so it cannot change the best policy (it shifts every value $V(x)$ by exactly $-|E(x)|$);
- **new largest tile:** the bonuses add up to about twice the largest tile reached, a graded version of the win.

In the code. Every reward is a function `r(board, after, points)` of the move: `board` is x, `after` is $y = M(x, u)$ and `points` is the score of the move. `count_free` is $|E(\cdot)|$, `milestone` is the first part (new largest tile) and `free_change` the third (change in free cells):

Code: src/shaped_rewards.py, lines 36-51: `count_free`, `milestone`, `free_change`

```
def count_free(board):
    n = 0
    for cell in board:
```

```
        if cell == 0:
            n += 1
    return n


def milestone(board, after):
    if max(after) > max(board):
        return 2 ** max(after)
    return 0


def free_change(board, after):
    return count_free(after) - count_free(board) - 1
```

and their sum. `2 + 2 * p4` is the second part; the variant `naive_nofree` leaves out the third, to test that it changes no decision:

Code: src/shaped_rewards.py, lines 61-71: `reward_fn`

```
def r(board, after, points):
    value = 0.0
    if name in ('naive', 'naive_nofree', 'sum'):
        value += 2 + 2 * p4
    if name in ('naive', 'naive_nofree', 'milestone'):
        value += milestone(board, after)
    if name in ('naive', 'free'):
        value += free_change(board, after)
    return value

return r
```

The exact DP confirms it on the 2x3 board: the optimal policy of the naive reward wins 0.9387 of the games, exactly as the score-optimal policy, against 0.9393 for the win-optimal one; with or without the free-cell part it picks the same move on all 68,340 boards. Trained with the naive reward, Sarsa on afterstates learns as fast as with the score (0.797 against 0.796 after 200,000 episodes). The naive reward works, but no better than the score, which the game provides and which has no weights to tune. We therefore use two rewards.

Win. +1 for the move that creates the target, which ends the game, and 0 for every other move:

$$r_{\text{win}}(x,u) = \mathbf{1}\big[\max_i y_i = K\big], \qquad y = M(x,u).$$

In RL the *return* of a game is the sum of the rewards collected until it ends, and the *value* $v_\pi(x)$ of a board is the expected return when the game continues from x with the policy π. With this reward the return is 1 for a won game and 0 for a lost one, and the expected value of something that is either 1 or 0 is the probability that it is 1. So $v_\pi(x)$ is the win probability of π from x, and the best value $v_*(x) = \max_\pi v_\pi(x)$ is the best win probability: exactly the $V^*(x)$ computed by the DP. The learners therefore aim at the very numbers of Part 1, which is why we can measure them exactly. This reward is the goal itself, but sparse: random play wins 2 games in 10,000 on the 2x3 board.

Score. The points of the move: every merge of two tiles 2^k creates a tile 2^{k+1} and pays its value,

$$r_{\text{score}}(x,u) = r(x,u) = \sum_{\text{merges of the move}} 2^{k+1},$$

where k is the exponent of the two tiles of each merge. The game does not stop at the target: it goes on until no legal move is left. The return is the points scored from the board x to the end of the game (the whole score, for a game counted from its start), so v_* is the V^*_{score} of the DP. It is dense and almost aligned with the goal: the score-optimal policy makes the win-optimal move in 97.9% of the decisions.

In the code. The points are computed line by line during the slide: two equal tiles of exponent `tiles[i]` merge into one of exponent `tiles[i] + 1`, and `points` grows by 2^{k+1}:

Code: src/game.py, lines 33-43: `slide_line`

```
tiles = [e for e in line if e]
merged, points, i = [], 0, 0
while i < len(tiles):
```

```
    if i + 1 < len(tiles) and tiles[i] == tiles[i + 1]:
        merged.append(tiles[i] + 1)
        points += 2 ** (tiles[i] + 1)
        i += 2
    else:
        merged.append(tiles[i])
        i += 1
return tuple(merged) + (0,) * (len(line) - len(merged)), points
```

As a reward function, the score just returns them:

Code: src/shaped_rewards.py, lines 54-57: `reward_fn`

```
def reward_fn(name, p4):
    """Returns r(board, after, points) for 'score' or one of REWARDS."""
    if name == 'score':
        return lambda board, after, points: points
```

The simulator pays the reward of each move. With the win reward, a move that creates the target pays 1 and ends the game, and every other move pays 0 (`reward_fn` is `None`); with any other reward it pays `reward_fn` of the move. The reward is computed before the new tile appears, so it depends on the move (x, u) alone:

Code: src/game.py, lines 226-240: `Env2048.step`

```
def step(self, action):
    """Returns (observation, reward, terminated, truncated, info)."""
    after, points = self.game.afterstate(self.board, action)
    if after == self.board:
        raise ValueError(f'illegal action {action!r}')
    if self.reward == 'win' and self.game.is_win(after):
        self.board = after
        return after, 1.0, True, False, {
            'won': True, 'afterstate': after, 'legal_actions': []}
    reward = (0.0 if self.reward_fn is None
              else float(self.reward_fn(self.board, after, points)))
    self.board = self.game.sample_spawn(after, self.rng)
    legal = self.legal_actions()
    return self.board, reward, not legal, False, {
        'won': False, 'afterstate': after, 'legal_actions': legal}
```

In the DP the win reward is not a stage reward: it is the terminal value 1 of an afterstate holding the target, in `afterstate_value` above. Every other reward is added to the value of the afterstate, in the line `q = w if r is None else w + r(board, after, points)` of `solve_optimal`:

Code: src/dynamic_programming.py, lines 41-45: `_stage_reward`

```
def _stage_reward(game, objective):
    """r(board, afterstate, points) of an objective, or None for 'win'
    (whose only reward is the terminal one, inside afterstate_value)."""
    _check(objective)
    return None if objective == 'win' else reward_fn(objective, game.p4)
```

The compiled engine of the 4x4 runs (`reward_mode` 0 is the win, 1 the score) pays the same two rewards:

Code: src/game_fast.py, lines 271-278: `env_step`

```
after, points, changed = move(x, action, rows, cols)
created = max_exp(after, rows, cols) >= target_exp
if reward_mode == 0 and created:
    return after, 1.0, True, after, True
nxt = spawn(after, rows, cols, p4)
terminal = legal_mask(nxt, rows, cols) == 0
reward = float(points) if reward_mode == 1 else 0.0
return nxt, reward, terminal, after, created
```

Every policy is measured by its exact win probability, whatever reward it learned from.

3.5 Prediction: TD(0), n-step TD, Monte Carlo

From here on we use the RL notation: S_t is the board before move t (the x_t of Part 1), A_t the move played, R_{t+1} its reward (the index is $t+1$ because the reward arrives with the next board), and an episode is one game. Prediction estimates the value v_π of a fixed policy π (the expected return from each board) from its episodes alone. We take the optimal policy and the win reward, so the true answer $v_\pi = V^*$ is known exactly. All methods use a constant step size α (how far each update moves the estimate) and $\gamma = 1$ (no discount).

TD(0) keeps a table V of estimates, one per board, and moves the estimate of a state towards the reward plus the estimate of the next state:

$$V(S_t) \leftarrow V(S_t) + \alpha\big[R_{t+1} + V(S_{t+1}) - V(S_t)\big].$$

n-step TD looks n steps ahead before using an estimate ($n = 2, 4, 8, 16$):

$$G_{t:t+n} = R_{t+1} + \cdots + R_{t+n} + V(S_{t+n}), \qquad V(S_t) \leftarrow V(S_t) + \alpha\big[G_{t:t+n} - V(S_t)\big].$$

Constant-α Monte Carlo uses the full return of the game, $G_t = R_{t+1} + R_{t+2} + \dots$ up to the end of the game, as target: the limit $n \to \infty$.

In the code. One function holds all three methods: $n = 1$ is TD(0), and $n =$ `MC_N` (a million, longer than any game) is Monte Carlo. The episodes are stored one after the other: `states[a + j]` is S_j and `rewards[a + j]` is R_{j+1} of the current episode, which has `T` moves. For each time `tau` the loop adds up the n rewards of $G_{\tau:\tau+n}$ and, if the game has not ended by then, adds $V(S_{\tau+n})$; near the end of the game the target is the plain return:

Code: src/tabular_prediction.py, lines 121-136: `nstep_td`

```
for e in range(n_ep):
    a = starts[e]
    T = starts[e + 1] - a
    for tau in range(T):
        end = min(tau + n, T)
        G = 0.0
        for j in range(tau, end):
            G += rewards[a + j]
        if tau + n < T:
            G += V[states[a + tau + n]]
        s = states[a + tau]
        old = V[s]
        new = old + alpha * (G - old)
        se += mu[s] * ((new - v_true[s]) ** 2 - (old - v_true[s]) ** 2)
        V[s] = new
    rms[e + 1] = math.sqrt(max(se, 0.0))
```

The error is the root mean square error against V^*, weighted by how often the policy visits each state (the RMSE of Exact learning curves, with `mu` the share of the visits that fall on each board). The function above keeps it up to date after every step (`se` is the sum under the square root); from scratch it is

Code: src/tabular_prediction.py, lines 140-142: `rms_error`

```
def rms_error(V, problem):
    """sqrt(VE) of the estimates V, computed from scratch."""
    return float(np.sqrt(np.sum(problem.mu * (V - problem.v_true) ** 2)))
```

On this game the ranking of the textbook random walk is reversed at the start: Monte Carlo is faster than TD(0), because the only reward comes at the end of a 33-move game and TD(0) passes it back one state per visit. With a longer budget an intermediate n wins (8-step TD).

3.6 Tabular control

Six control methods of the course, each keeping a table of values and improving the policy as the values change. $Q(S_t, A_t)$ is the table's estimate of the value of playing A_t on S_t: the expected return of that move and of the moves after it. All of them choose their moves ε-greedily: the move with the largest Q, except that with probability ϵ they play a random legal move (see Exploration).

In the code. The table `Q` maps a board to an array of four values, one per move, in the order down, left, right, up; `self.q(s)` is the row of the board `s`. Each method is the `episode` method of its class: `s, a, r` are S_t, A_t, R_{t+1}, and `s2, a2` are S_{t+1}, A_{t+1}. At the end of the game the target is the reward alone, since there is no next move. The parameter γ is `g` (always 1 here).

Monte Carlo control (on-policy, first visit, ε-soft): after each game, $Q(S_t, A_t)$ moves to the average of the observed returns G_t. Since no state repeats within a game, first-visit and every-visit coincide.

The returns are accumulated backwards from the end of the game, $G_t = R_{t+1} + G_{t+1}$. With `alpha=None` the step is $1/N(S_t, A_t)$, which keeps Q equal to the average of the returns:

Code: src/tabular_control.py, lines 306-316: `MCControl.episode`

```
G = 0.0
# states never repeat within an episode: every visit is a first visit
for s, a, r in reversed(trajectory):
    G = r + g * G
    qs = self.q(s)
    if alpha is None:
        self.N[(s, a)] += 1
        qs[a] += (G - qs[a]) / self.N[(s, a)]
    else:
        qs[a] += alpha * (G - qs[a])
return G, info['won'], len(trajectory)
```

Sarsa (on-policy TD) uses the next move actually played:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big].$$

Code: src/tabular_control.py, lines 180-191: `Sarsa.episode`

```
while True:
    s2, r, done, _, info = env.step(ACTIONS[a])
    G += r
    steps += 1
    qs = self.q(s)
    if done:
        qs[a] += alpha * (r - qs[a])
        return G, info['won'], steps
    legal = _legal(info)
    a2 = epsilon_greedy(self.q(s2), legal, eps, rng)
    qs[a] += alpha * (r + g * self.Q[s2][a2] - qs[a])
    s, a = s2, a2
```

Q-learning (off-policy TD) uses the best next move, so it learns the optimal values q_* (the value of each move when the best policy plays afterwards) whatever the exploration:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\big].$$

The maximum runs over the legal moves of S_{t+1} only:

Code: src/tabular_control.py, lines 202-214: `QLearning.episode`

```
while True:
    a = epsilon_greedy(self.q(s), legal, eps, rng)
    s2, r, done, _, info = env.step(ACTIONS[a])
    G += r
    steps += 1
    qs = self.q(s)
    if done:
        qs[a] += alpha * (r - qs[a])
        return G, info['won'], steps
    legal = _legal(info)
    q2 = self.q(s2)
    qs[a] += alpha * (r + g * max(q2[b] for b in legal) - qs[a])
    s = s2
```

Expected Sarsa uses the average over the next moves of the ε-greedy policy ($\pi(a \mid S_{t+1})$ is the probability that the policy plays a on S_{t+1}), a target with less variance than Sarsa's:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \sum_a \pi(a \mid S_{t+1})\, Q(S_{t+1}, a) - Q(S_t, A_t)\Big].$$

`expect` is the sum over the legal moves a of $\pi(a \mid S_{t+1})\, Q(S_{t+1}, a)$:

Code: src/tabular_control.py, lines 225-239: `ExpectedSarsa.episode`

```
while True:
    a = epsilon_greedy(self.q(s), legal, eps, rng)
    s2, r, done, _, info = env.step(ACTIONS[a])
    G += r
    steps += 1
    qs = self.q(s)
    if done:
        qs[a] += alpha * (r - qs[a])
        return G, info['won'], steps
    legal = _legal(info)
    q2 = self.q(s2)
    expect = sum(p * q2[b] for b, p in
                 epsilon_greedy_probs(q2, legal, eps).items())
    qs[a] += alpha * (r + g * expect - qs[a])
    s = s2
```

The probabilities of the ε-greedy policy: $\epsilon/|U(x)|$ for every legal move, plus $1-\epsilon$ for the greedy one:

Code: src/tabular_control.py, lines 102-107: `epsilon_greedy_probs`

```
def epsilon_greedy_probs(q, legal, eps):
    """{action: probability} of the epsilon-greedy policy on legal."""
    base = eps / len(legal)
    probs = {a: base for a in legal}
    probs[greedy(q, legal)] += 1.0 - eps
    return probs
```

n-step Sarsa ($n = 3$) is Sarsa with the target $R_{t+1} + R_{t+2} + R_{t+3} + Q(S_{t+3}, A_{t+3})$. The code follows the pseudocode of Sutton & Barto §7.2: at time `t` it updates the pair of time `tau` $= t - n + 1$, whose n rewards are now known; `G` is the target, without the last term when the game ends within n moves:

Code: src/tabular_control.py, lines 269-277: `NStepSarsa.episode`

```
tau = t - n + 1
if tau >= 0:
    G = 0.0
    for i in range(tau + 1, min(tau + n, T) + 1):
        G += g ** (i - tau - 1) * rewards[i]
    if tau + n < T:
        G += g ** n * self.Q[states[tau + n]][actions[tau + n]]
    qs = self.q(states[tau])
    qs[actions[tau]] += alpha * (G - qs[actions[tau]])
```

Sarsa on afterstates. Two different moves from two different boards can lead to the same afterstate, and from there the future is the same. So we keep one value per afterstate and write the value of a move as

$$Q(x, u) = r(x, u) + V\big(M(x, u)\big).$$

This is exact, not an approximation: the true value of a move depends on (x, u) only through its points and its afterstate. The agent uses the slide rule M but never the probabilities of the new tile. With this Q the reward cancels in the Sarsa error, and the update becomes an update of one afterstate value ($Y_t = M(S_t, A_t)$, target 0 at game over):

$$V(Y_t) \leftarrow V(Y_t) + \alpha\big[r(S_{t+1}, A_{t+1}) + V(Y_{t+1}) - V(Y_t)\big].$$

In the code. The table `V` holds one value per afterstate, and the value of a move is $Q(x, u) = r(x, u) + V(M(x, u))$. With the win reward, a move that creates the target is worth 1, the only reward of the game:

Code: src/tabular_control.py, lines 350-358: `AfterstateSarsa.action_values`

```
def action_values(self, board, moves):
    """{action name: r + V[afterstate]} over the legal moves."""
    q = {}
    for action, after, points in moves:
        if self.win_mode:
            q[action] = 1.0 if self.game.is_win(after) else self.v(after)
        else:
            q[action] = self.r(board, after, points) + self.gamma * self.v(after)
    return q
```

The update: `after` is Y_t, the next move gives `after2` $= Y_{t+1}$ and `points2`, and the target is the value of that next move, $r(S_{t+1}, A_{t+1}) + V(Y_{t+1})$ (0 when the next board is game over, a few lines above):

Code: src/tabular_control.py, lines 385-392: `AfterstateSarsa.episode`

```
moves2 = self.game.moves(s2)
action2, after2, points2 = self._choose(s2, moves2, eps, rng)
if self.win_mode:
    target = 1.0 if self.game.is_win(after2) else self.v(after2)
else:
    target = self.r(s2, after2, points2) + self.gamma * self.v(after2)
self.V[after] = self.v(after) + alpha * (target - self.v(after))
s, action, after = s2, action2, after2
```

Every update improves all the moves, from all the boards, that lead to the same afterstate, which is why this method learns fastest. The same update with a linear approximation of V is the core of the strongest published 2048 players (Szubert & Jaśkowski 2014 and their successors).

3.7 Exploration

- **ε-greedy:** the agent plays a uniformly random legal move with probability ϵ, and otherwise the move with the largest Q (ties are broken in the order down, left, right, up, so an agent with all-zero values plays the corner rule).
- **Optimistic initial values:** start every Q at 1, the largest possible win probability. Every untried move looks promising until it has been tried, so the agent explores every move once and then follows the best. With the sparse win reward this removes the dead start of a learner that has never seen a win.
- **Step-size and exploration schedules:** we compare constant α and ε with values that decrease linearly during training (ε from 0.3 to 0.01, α from 0.2 to 0.02), and try a grid of constant values.

In the code. ε-greedy, with ties going to the first move in the order down, left, right, up (`legal` lists the legal moves in that order):

Code: src/tabular_control.py, lines 87-99: `greedy`, `epsilon_greedy`

```
def greedy(q, legal):
    """First maximiser (in ACTIONS order) among the legal action indices."""
    best, best_q = legal[0], q[legal[0]]
    for a in legal[1:]:
        if q[a] > best_q:
            best, best_q = a, q[a]
    return best


def epsilon_greedy(q, legal, eps, rng):
    if eps > 0.0 and rng.random() < eps:
        return legal[rng.randrange(len(legal))]
    return greedy(q, legal)
```

Optimistic initial values: a board met for the first time gets a row of four values equal to `q_init` (1 for optimism, 0 otherwise):

Code: src/tabular_control.py, lines 132-136: `TabularAgent.q`

```
def q(self, s):
    row = self.Q.get(s)
    if row is None:
        row = self.Q[s] = np.full(4, self.q_init)
```

```
    return row
```

The linear schedule, a function of the episode number `t`:

Code: src/tabular_control.py, lines 61-66: `linear`

```
def linear(start, end, n_episodes):
    """Linear from start (episode 1) to end (episode n_episodes), then end."""
    def f(t):
        frac = min(1.0, (t - 1) / max(1, n_episodes - 1))
        return start + (end - start) * frac
    return f
```

3.8 Exact learning curves

This is how every RL result on the small boards is measured. A learning curve shows how good an agent is as its training goes on. It is usually drawn by stopping every so often, letting the agent play some test games and averaging their results, so the curve is noisy: a lucky batch of test games looks like progress. On the small boards no test games are needed, because the DP evaluates any policy exactly. So at regular checkpoints we take the agent's greedy policy (on every board the move with the largest Q, without the random moves of exploration) and measure two things exactly.

How well it plays: the win probability and the expected score of the greedy policy, computed by the policy evaluation above. These are the numbers in the tables of results.

How well it knows the values: on a board x the agent's estimate of the value is its best move value, $\hat{V}(x) = \max_u Q(x, u)$ (the hat marks an estimate), and the true value is the $V^*(x)$ of the DP ($V^*_{\text{score}}(x)$ for an agent trained on the score). The errors $\hat{V}(x) - V^*(x)$ on all the boards are summarised in one number, a weighted root mean square error:

$$\text{RMSE} = \sqrt{\sum_x d(x) \left(\hat{V}(x) - V^*(x)\right)^2}.$$

Read it from the inside out: square each error, so that errors up and down do not cancel; average the squares over the boards, each board x with weight $d(x)$; take the square root, to come back to the units of the values. An RMSE of 0.02 on the win probability means estimates off by about 2 percentage points. The weight $d(x)$ is the share of the optimal player's decisions taken on the board x (computed exactly with the DP), and the weights add up to 1. Most boards are almost never reached by a good player, and an error there costs nothing in play; the weights make the error count where the game is actually played. The sum runs over the boards the agent has met, since it has no estimate for the others.

Neither measurement uses random games, so the curves have no measurement noise: every change in them is a change in what the agent has learned.

In the code. `ExactEvaluator.evaluate` runs the policy evaluation of Part 1 on the greedy policy, from the start of the game, for the win probability and the expected score. Then, on every board of the optimal player, it adds up the squared errors weighted by `visits`, the expected number of visits of the optimal player to the board (from `dp.occupancy`). Dividing by `seen`, the total weight of the boards the agent has met, turns these weights into the shares $d(x)$ that add up to 1:

Code: src/tabular_control.py, lines 483-514: `ExactEvaluator.evaluate`

```
out = {'p_win': dp.start_value(
    g, dp.evaluate_policy(g, pi, 'win', self.layers_win))}
if self.V_score is not None:
    out['score'] = dp.start_value(
        g, dp.evaluate_policy(g, pi, 'score', self.layers_score))
refs = {'win': self.V_win, 'score': self.V_score,
        **getattr(self, 'V_extra', {})}
V_ref = refs.get(agent.reward)
total = seen = agree = 0.0
err_w = err = 0.0
n_seen = 0
for board, visits in self.mu.items():
    moves = g.moves(board)
    if not moves:
```

```
            continue
        total += visits
        a = agent.greedy_action(board, moves)
        if a is None:
            continue
        seen += visits
        if a in self.optimal[board]:
            agree += visits
        if V_ref is not None:
            d = agent.state_value(board, moves) - V_ref[board]
            err_w += visits * d * d
            err += d * d
            n_seen += 1
    out['unseen'] = 1.0 - seen / total
    out['agree'] = agree / seen if seen else 0.0
    if V_ref is not None and n_seen:
        out['rmse_w'] = math.sqrt(err_w / seen)
        out['rmse'] = math.sqrt(err / n_seen)
```

3.9 Final test: deep Q-network

Outside the methods of the course, the RL part ends with a deep Q-network built as in the PyTorch tutorial: Q-learning with a neural network $Q(x, u; \theta)$, trained on random minibatches of past transitions (experience replay) towards targets computed by a slowly updated copy of the network (target network θ'):

$$\text{loss} = \text{Huber}\Big(r + \gamma \max_{u'} Q(x', u'; \theta') - Q(x, u; \theta)\Big).$$

Here (x, u, r, x') is a stored transition (board, move, points, next board), $\gamma < 1$ is a discount that weighs later rewards less, and the Huber loss is the squared error for small errors and the absolute error for large ones, so a few large errors do not dominate the gradient. The input is the one-hot encoding of the exponent of every cell; illegal moves are masked.

The reward is the score, divided by the target tile so that the returns are of order one:

$$r = \frac{r_{\text{score}}(x, u)}{2^K}.$$

Scaling every reward by the same number scales every Q by it too, so the greedy policy does not change. In the code, when a transition is stored in the replay memory (`self.reward_scale` is the target tile, 64 on 2x3 and 2048 on 4x4):

Code: src/dqn_tutorial.py, lines 202-202: `DQNAgent.train_episode`

```
self.memory.push(obs, a, points / self.reward_scale, obs2, mask2, done)
```

In the code. `s, a, r, s2` is a minibatch of transitions (x, u, r, x'); `m2` marks the legal moves of x', so the maximum runs over them only, and `done` sets the target to r alone when x' is game over. `F.smooth_l1_loss` is PyTorch's Huber loss:

Code: src/dqn_tutorial.py, lines 162-170: `DQNAgent.optimize_model`

```
s, a, r, s2, m2, done = self.memory.sample(BATCH_SIZE, self.rng, self.device)
state_action_values = self.policy_net(s).gather(1, a[:, None])
with torch.no_grad():
    q2 = self.target_net(s2)
    q2[~m2] = -float('inf')
    next_values = q2.max(1).values
    next_values[done] = 0.0
expected = next_values * GAMMA + r
loss = F.smooth_l1_loss(state_action_values, expected[:, None])
```

</details> It is tested on the 2x3 board, where it is evaluated exactly, and on the original 4x4 game. With the tutorial's schedule (one gradient step per move) it is slow: 2,314 points after 10,000 games of the 4x4 game, below the rules of thumb. Playing 1,024 games at once on a GPU, with a replay memory of a million moves (as in the original DQN), it reaches 0.871 of games won on 2x3 and 8,862 points on 4x4 after 100,000 games; with a convolutional network over the pairs of neighbouring cells, 17,485 points and the 2048 tile in

36% of the games. Neither curve is still rising at 100,000 games, and the n-tuple players of the literature, which are given the slide rule through the afterstates, stay far ahead (2048 in over 97% of the games).

3.10 Extra: Sarsa with n-tuple features

Also outside the methods of the course: we re-implemented the n-tuple players of the literature (Szubert & Jaśkowski 2014), the method behind the strongest published 2048 players, to see how it works. It is Sarsa on afterstates with the table over whole boards replaced by a sum of small tables:

$$V(y;w) = \sum_{m=1}^{M} w_m\big[\text{index}_m(y)\big].$$

An n-tuple is a fixed group of n cells, for example a row. For each of the M tuples, $\text{index}_m(y)$ reads the exponents found in its cells as a single number, and w_m is the tuple's own table of weights, one weight for every possible content of those cells. The value of an afterstate is the sum of the M weights its tuples point to. Two boards with the same content in a row share the weight of that row, so what is learned on one board carries over to boards never seen: the generalisation that the tabular methods lack on the 4x4 game. The update is the one of Sarsa on afterstates, applied to the M weights that Y_t points to, each moved by α/M times the error (semi-gradient Sarsa, Sutton & Barto §10.1):

$$w_m\big[\text{index}_m(Y_t)\big] \leftarrow w_m\big[\text{index}_m(Y_t)\big] + \frac{\alpha}{M}\big[r(S_{t+1},A_{t+1}) + V(Y_{t+1};w) - V(Y_t;w)\big], \qquad m = 1,\dots,M.$$

In the code. The board is a 64-bit integer with 4 bits per cell, and `fb.nibble(x, c)` reads the exponent of the cell `c`. Row `m` of `cells` lists the cells of tuple m; the loop over `j` packs their exponents into `idx`, which is $\text{index}_m(y)$, and `W[owner[m], idx]` is the weight $w_m[\text{index}_m(y)]$. Tuples that are rotations or reflections of each other share one table (`owner[m]`):

Code: src/ntuple_network.py, lines 155-172: `evaluate`, `update`

```
@njit(cache=True)
def evaluate(x, W, cells, owner):
    v = 0.0
    for m in range(cells.shape[0]):
        idx = 0
        for j in range(cells.shape[1]):
            idx |= fb.nibble(x, cells[m, j]) << (4 * j)
        v += W[owner[m], idx]
    return v


@njit(cache=True)
def update(x, W, cells, owner, delta):
    for m in range(cells.shape[0]):
        idx = 0
        for j in range(cells.shape[1]):
            idx |= fb.nibble(x, cells[m, j]) << (4 * j)
        W[owner[m], idx] += delta
```

The update, inside one game: `q2` is the value of the next move, $r(S_{t+1},A_{t+1}) + V(Y_{t+1};w)$, and each of the M weights of `after` $= Y_t$ moves by α/M times the error:

Code: src/ntuple_network.py, lines 254-258: `play_td`

```
if learn:
    target = q2 if a2 >= 0 else 0.0
    # alpha is the step on V: every active weight moves by alpha / M
    delta = alpha / cells.shape[0] * (target - evaluate(after, W, cells, owner))
    update(after, W, cells, owner, delta)
```

We ran it on the 2x3, 2x4 and 3x3 boards, where the DP measures every learned policy exactly, and on the 4x4 game, where it is the extra n-tuple player of the policy viewer. The code is in `src/ntuple_network.py`, the scripts under "Extra" in Reproducing the results.

4 Main results

	2x3 board, target 64
Reachable states	27,022 in 55 stages
Optimal policy (exact DP)	wins 93.93% of the games; maximum expected score 480.3
Corner / greedy / random rule	0.07% / 0.13% / 0.02%
Best tabular learner, win reward with optimism	Q-learning 0.875; Sarsa on afterstates 0.848
Best tabular learner, score reward	Sarsa on afterstates 0.796 (0.85 with decaying schedules)
DQN (final test), score reward	0.812 with the tutorial's schedule (20,000 games); 0.871 with many games on the GPU (200,000)
Prediction after 20,000 games (error against V^*)	8-step TD 0.080, Monte Carlo 0.085, TD(0) 0.114

	4x4 board, target 2048: mean score, tiles reached
Corner / greedy rule	2,564 / 3,027 points; 512 in 5% / 12% of the games
Tabular methods, settings of the 2x3 runs (200,000 episodes)	from 2,163 (Sarsa on afterstates) to 2,390 points (Q-learning), below the corner rule; 50-74% of the moves on boards never met in training
DQN, tutorial's schedule (10,000 games)	2,314 points; 512 in 4%, never 1024
DQN on the GPU, perceptron (100,000 games)	8,862 points; 1024 in 41%, 2048 in 3%
DQN on the GPU, convolutional network (100,000 games)	17,485 points; 1024 in 73%, 2048 in 36%, 4096 in 1.3%

Win probabilities on 2x3 are exact and averaged over the seeds; scores on 4x4 are means over 1,000 games (10,000 for the rules) and over the seeds (five for the tabular methods, three for the DQN, two for the convolutional network).

The tabular methods on the 4x4 game. The six methods run on the original game unchanged, with the settings of the 2x3 runs (score reward, alpha 0.1, MC control with sample averages, epsilon 0.1, 200,000 episodes, 5 seeds). `src/tabular_control_fast.py` holds the same algorithms as `src/tabular_control.py` with the table in a hash table keyed by the 64-bit board; on 2x3, with the same settings, its exact win probabilities agree with those of `src/tabular_control.py` within the spread of the seeds (`results/tabular_4x4/port_check_2x3.csv`). After 200,000 episodes each table holds 23-28 million boards, nearly all met only once, and in a new game 69-74% of the moves fall on boards never met in training (50% for Sarsa on afterstates, which needs one known afterstate out of four). There the table has no value and the corner rule plays: all six methods stay below the corner rule itself (2,564 points), and more training does not raise the score. More exploration makes it worse: with epsilon 0.2 and 0.3 every method scores less and meets fewer of the boards of its own greedy play, because the random moves scatter the training games over boards that the greedy policy never meets. A table cannot generalise from one board to another: this is the curse of dimensionality already visible in the state counts of Part 1, and the reason why the final test needs a network that generalises.

Tabular method on 4x4	**epsilon 0.1**	**epsilon 0.2**	**epsilon 0.3**
Monte Carlo control	2,287 (74%)	2,120 (82%)	2,129 (86%)
Sarsa	2,372 (69%)	2,319 (80%)	2,279 (84%)
Expected Sarsa	2,367 (69%)	2,292 (79%)	2,289 (84%)
Q-learning	2,390 (69%)	2,329 (80%)	2,319 (84%)
3-step Sarsa	2,355 (72%)	2,207 (81%)	2,190 (85%)
Sarsa on afterstates	2,163 (50%)	1,958 (57%)	1,901 (62%)

Mean score of the greedy policy (1,000 games, mean over 5 seeds) and, in brackets, the share of its moves on boards never met in training; the corner rule scores 2,564 points, the greedy rule 3,027.

Exact DP solves boards up to 3x3 (4.8 million states); the 4x4 game has up to $11^{16} \approx 4.6 \cdot 10^{16}$ states. On the small boards the corner rule is far from optimal (0.07% against 93.9% on 2x3/64, 43% against 100% on 3x3/64): on a cramped board what matters is whether a merge survives the next random tile, not where the largest tile sits. Luck matters too: with only 2s the 2x3 game is always won, and at $p_4 = 0.5$ even the optimal player wins only 72.7%.

5 Repository

5.1 Setup

Python 3.10 or later. From this folder:

```
python -m venv .venv
.venv\Scripts\activate          # Windows;  source .venv/bin/activate on Linux/macOS
pip install -r requirements.txt
python src/tests.py             # about one minute: rules, DP, engines, learning, DQN smoke test
```

5.2 Code

The modules are in `src/`, the scripts that produce the results in `experiments/` (they import the modules from `src/` by themselves).

File	Content
`src/game.py`	Rules of the game on an R x C board (pure Python, the reference implementation), `Env2048` with the Gymnasium `reset`/`step` interface
`src/dynamic_programming.py`	Exact DP: reachable states by stage, backward recursion, value iteration (total cost; Jacobi or Gauss-Seidel by stage), exact policy evaluation, occupancy, Monte Carlo simulation
`src/game_fast.py`	The same rules on a 64-bit bitboard compiled with numba (about 50x faster); `src/tests.py` checks it against `src/game.py` move by move
`src/tabular_prediction.py`	Tabular prediction of a fixed policy: TD(0), n-step TD and constant-alpha Monte Carlo, with the error against the exact values
`src/shaped_rewards.py`	The rules-of-thumb ("naive") reward: a bonus for each new largest tile, the growth of the tile sum, the change of free cells; usable as a reward name wherever `'win'`/`'score'` are
`src/tabular_control.py`	Tabular control agents on `Env2048`: MC control, Sarsa, Q-learning, Expected Sarsa, n-step Sarsa, Sarsa on afterstates; `ExactEvaluator` computes the exact value of a learned policy with the DP
`src/tabular_control_fast.py`	The same six tabular methods on the bitboard engine, with the table in a hash table keyed by the 64-bit board, for the 4x4 game (over a million moves per second)
`src/dqn_tutorial.py`	Final test: DQN as in the PyTorch tutorial (replay memory, target network, Huber loss, AdamW), with legal-move masking; needs PyTorch
`src/dqn_gpu.py`	The same agent on the GPU with 1,024 games at once (rules in the numba engine), a replay memory of a million moves, the perceptron or a convolutional network (`PairCNN`); needs PyTorch with CUDA
`src/ntuple_network.py`	Extra, outside the main line of the project: a re-implementation of the n-tuple players of the literature (semi-gradient Sarsa on afterstates with binary n-tuple features), kept for exploration and used by the 4x4 policy viewer
`src/visualize.py`	Animations of the policies playing (`python src/visualize.py -h`)
`src/tests.py`	Checks of the rules, the DP, the two engines and the learning methods (`python src/tests.py`)
`viewer/`	Policy viewer, online at https://edoardocaproni.github.io/2048/ or opened locally (`viewer/index.html`): the best policy of every method plays. On the 2x3 board every policy is a table over the decision states, shown with the exact optimal win probability of every position and move; on the 4x4 board the page plays the rules of thumb, the six tabular methods (the 50,000 boards each table met most often in training, the corner rule elsewhere), the three DQNs of the final test (the convolutional network with its weights in half precision) and an extra n-tuple player. Data written by `experiments/export_viewer.py`; the page loads the 4x4 agents (8 MB) only when that board is chosen, and the tables (5 MB) only when one of them is
`.github/workflows/pages.yml`	Publishes `viewer/` as the GitHub page at every push to `main` that changes it. One-time setup by the owner of the repository: Settings > Pages > Build and deployment > Source: GitHub Actions

5.3 Reproducing the results

All the scripts write to `results/`; run them from this folder with the venv active.

```
python experiments/dp_figures.py            # Part 1 figures and tables (add 'big' for the 3x3 board, 5 min)
```

```
python experiments/dp_value_iteration.py    # value iteration against the backward recursion (2 min)
python experiments/bench_engine.py          # throughput of the two engines
python experiments/exp_reward.py dp         # optimal policy of each reward, exact (a few minutes)
python experiments/exp_reward.py tabular    # Sarsa on afterstates with the naive reward, 5 seeds (15 min)
python experiments/plot_reward.py
python experiments/exp_prediction.py        # TD(0), n-step TD and MC prediction, 50 runs (under a minute)
python experiments/exp_tabular_2x3.py       # tabular methods, 90 runs x 200k episodes (about 1 h on 26 cores)
python experiments/plot_tabular.py
python experiments/exp_tabular_grid.py      # alpha x epsilon grid and decaying schedules (130 runs)
python experiments/plot_schedule.py
python experiments/exp_tabular_4x4.py check # the fast tabular code on 2x3, exact P(win) against
    tabular_control.py (a minute)
python experiments/exp_tabular_4x4.py       # the six tabular methods on 4x4, 5 seeds x 200k episodes (3 min, 3
     GB of RAM per process)
python experiments/exp_tabular_4x4.py eps=0.2,0.3   # the same with more exploration (6 min)
python experiments/plot_tabular_4x4.py
python experiments/eval_4x4.py              # the rules of thumb on the 4x4 board (and any n-tuple checkpoints
    present)
python experiments/exp_dqn.py               # DQN final test, tutorial's schedule: 3 seeds on 2x3 and on 4x4,
    on the CPU (about 1 h)
python experiments/exp_dqn_gpu.py           # DQN on the GPU, perceptron: 3 seeds on 2x3 and on 4x4 (about 20
    min)
python experiments/exp_dqn_gpu.py 4x4 net=cnn games=100000 eval_every=5000 seeds=0,1   # convolutional network
    (about 1.5 h)
python experiments/plot_dqn_gpu.py
python experiments/export_viewer.py         # data of the viewer: retrains the best tabular policies, exports
    the DQN networks (a few minutes; the 4x4 n-tuple player trains for about 1 h the first time)
```

Extra (exploration of the n-tuple players of the literature):

```
python experiments/exp_ntuple_small.py      # semi-gradient Sarsa, n-tuple features, on 2x3, 2x4, 3x3 with
    exact validation
python experiments/plot_ntuple_small.py
python experiments/exp_4x4.py               # 4x4 training grid, 300k games per run (hours; '100000 rcs' for a
    short run)
python experiments/plot_4x4.py
```